

SQL INJECTION :

28 March 2025 09:51

1. With Statement:

```
package jdbc;
import java.sql.*;

public class SqlPractice {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/pi";
        String user = "root";
        String password = "ajstyles";

        Connection connection = null;
        Statement stmt = null; // Using Statement instead of PreparedStatement
        ResultSet rs = null;

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            connection = DriverManager.getConnection(url, user, password);

            // Simulating user input (vulnerable to injection)
            String userInput = "HR' OR '1'='1"; // Malicious input
            String query = "SELECT * FROM branch WHERE name = '" + userInput + "'";

            stmt = connection.createStatement();
            rs = stmt.executeQuery(query);

            while (rs.next()) {
                int id = rs.getInt(1);
                String name = rs.getString(2);
                System.out.println(id + " : " + name);
            }

        } catch (SQLException | ClassNotFoundException e) {
            e.printStackTrace();
        } finally {
            try {
                if (rs != null) rs.close();
                if (stmt != null) stmt.close();
                if (connection != null) connection.close();
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
    }
}
```

2. With Prepared Statement:

```
package jdbc;
import java.sql.*;
```

```

public class SqlWithoutInjectionPractice {
    public static void main(String[] args) {

        System.out.println("'1'='1");

        String url = "jdbc:mysql://localhost:3306/pi";
        String user = "root";
        String password = "ajstyles";

        Connection connection = null;
        PreparedStatement stmt = null; // Use PreparedStatement
        ResultSet rs = null;

        try {
            Class.forName("com.mysql.cj.jdbc.Driver");
            connection = DriverManager.getConnection(url, user, password);

            // Simulating user input
            String userInput = "HR' OR '1'='1"; // Malicious input

            // Secure parameterized query
            String query = "SELECT * FROM branch WHERE name = ?";
            stmt = connection.prepareStatement(query);
            stmt.setString(1, userInput); // Safely binds the input

            rs = stmt.executeQuery();

            while (rs.next()) {
                int id = rs.getInt(1);
                String name = rs.getString(2);
                System.out.println(id + " : " + name);
            }

        } catch (SQLException | ClassNotFoundException e) {
            e.printStackTrace();
        } finally {
            try {
                if (rs != null) rs.close();
                if (stmt != null) stmt.close();
                if (connection != null) connection.close();
            } catch (SQLException e) {
                e.printStackTrace();
            }
        }
    }
}

```

3. Batch Processing :

```

package jdbc;

import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

```

```

public class InsertDemo {

    private final static String url = "jdbc:mysql://localhost:3306/pi";
    private final static String userName = "root";
    private final static String password = "ajstyles";

    public static void main(String[] args) throws SQLException {

        try {

            // 1. step

            Class.forName("com.mysql.cj.jdbc.Driver");

        } catch (ClassNotFoundException e) {
            e.printStackTrace();
        }

        // 2nd step :

        Connection connection = DriverManager.getConnection(url, userName, password);
        System.out.println(connection);

        // step 3 .

        PreparedStatement ps = connection.prepareStatement("insert into branch
values(? , ?)");

        // in this case we use the positional parameter use.

        ps.setInt(1, 16);
        ps.setString(2, "HR");

        ps.addBatch();

        ps.setInt(1, 17);
        ps.setString(2, "IT");

        ps.addBatch();

        ps.setInt(1, 19);
        ps.setString(2, "SC");

        ps.addBatch();

        int[] execute = ps.executeBatch();

        for (int i : execute) {

            if (i > 0) {
                System.out.println("success");
            } else {
                System.out.println("failed");
            }
        }
    }
}

```

```
        connection.close();  
        ps.close();  
    }  
}
```